0025. Reverse Nodes in k-Group

d | Category: Linked List | LeetCode: https://leetcode.com/problems/reverse-node

Problem

- Reverse the list k nodes at a time; a final group shorter than k stays as is. O(1) extra memory.
- Example: 1..5, k = 2 → 2,1,4,3,5; k = 3 → 3,2,1,4,5.

Intuition

- Process groups one by one: check that k nodes exist, reverse exactly those k, then reconnect with the previous group's tail and the next group's head.
- GroupPrev is the node before the current group; after reversal, the group's original first node becomes the new groupPrev.

Brute Force

- Copy values into an array, reverse chunks, rebuild: O(n) extra space.

Optimal Approach

- Dummy → head; groupPrev = dummy.
- Kth = walk k steps from groupPrev; if null, stop.
- GroupNext = kth.next. Reverse nodes from groupPrev.next up to kth, with prev starting at groupNext.
- First = groupPrev.next; groupPrev.next = kth; groupPrev = first.

Worked Example

```
1,2,3,4,5 with k = 2.

group [1,2] -> 2,1   list: 2,1,3,4,5   groupPrev=1
group [3,4] -> 4,3   list: 2,1,4,3,5   groupPrev=3
group [5] has < 2 nodes -> stop
```

Java Solution

```java
public class ReverseNodesInKGroup {
    public ListNode reverseKGroup(ListNode head, int k) {
        ListNode dummy = new ListNode(0, head), groupPrev = dummy;
        while (true) {
            ListNode kth = groupPrev;
            for (int i = 0; i < k && kth != null; i++) kth = kth.next;
            if (kth == null) break;
            ListNode groupNext = kth.next, prev = groupNext, cur = groupPrev.next;
            while (cur != groupNext) {
                ListNode nx = cur.next;
                cur.next = prev;
                prev = cur;
                cur = nx;
            }
            ListNode first = groupPrev.next;
            groupPrev.next = kth;
            groupPrev = first;
        }
        return dummy.next;
    }
}
```

Complexity

- Time: O(n). Space: O(1).

Pitfalls

- Initialising prev = groupNext makes the reversed tail point to the next group automatically.